

Positioning

Python-прототип пассивной акустической локализации источника звука по TDOA для компланарной решетки из четырех направленных микрофонов.

Проект используется как практическая часть дипломной работы: он формирует воспроизводимый программный стенд для синтетических сигналов, предобработки, проверки задержек, локализации направления/координат, сопровождения цели и протоколирования результатов ПМИ.

Быстрый запуск

Команды выполняются из папки `positioning`.

```
python -m pip install -e ".[dev,docs]"
```

```
python .\tools\demo.py
python .\tools\validate.py
python -m pytest
python .\tools\export_docs_to_pdf.py
```

Расширенный сценарий с файлами, журналами и ПМИ:

```
positioning generate-synthetic --output .\data\synthetic\signal.csv --source-kind pulse
--reflection-delay 0.002 --reflection-gain 0.2
positioning generate-synthetic --format wav --output .\data\synthetic\signal.wav --source-kind pulse
positioning generate-trajectory --output .\data\synthetic\trajectory_demo.csv --duration 1.2
--point-count 12 --noise-std 0.001
positioning inspect-audio --input .\data\synthetic\signal.wav --require-metadata
positioning preprocess --input .\data\synthetic\signal.csv --output-dir .\data\processed
positioning preprocess --input .\data\synthetic\trajectory_demo.csv --output-dir .\data\processed_demo
positioning localization --input .\data\synthetic\trajectory_demo.csv --mode position
--show-visualization --trajectory-max-windows 45
positioning localize-audio --input .\data\synthetic\signal.csv --mode direction --max-windows 1
--csv-log .\data\logs\audio_localization.csv --json-output .\data\logs\audio_final_result.json
positioning demo --json-output .\data\logs\demo_result.json
positioning track-demo --csv-log .\data\logs\tracking.csv --json-output .\data\logs\final_result.json
--inject-low-confidence-jump
positioning pmi --output-dir .\data\pmi --repeat-count 100
```

Основная структура

- `src/positioning/` - активный Python-пакет.
- `configs/default.json` - конфигурация решетки, аудио, среды и режима локализации.
- `tools/demo.py` - удобная обертка для демонстрации.
- `tools/validate.py` - удобная обертка для синтетического валидатора.
- `tests/` - pytest-набор.
- `docs/` - подробная документация проекта.
- `docs/pdf/` - PDF-экспорт документации.
- `legacy_cpp/` - архив старого C++-прототипа, не участвует в активной сборке.
- `data/` - генерируемые сигналы, журналы и протоколы, не отслеживаются Git.

Что реализовано

- Формальные модели данных: микрофонная решетка, среда, аудиофрейм, TDOA-оценка, результат локализации.
- JSON-конфигурация без внешних зависимостей.

- Компланарная квадратная решетка по умолчанию в плоскости $z=0$.
- Генерация синтетических 4-канальных сигналов с шумом, ослаблением, выбросом и отраженным путем.
- Генерация синтетической траектории движущегося источника с массивом trajectory в metadata.
- Чтение и запись 4-канальных CSV/WAV-файлов и sidcar-метаданных.
- Разбиение на окна, удаление DC, нормализация, Hann-окно, частотная маска и диагностика информативности.
- Проверка TDOA: физическая допустимость, транзитивность, качество пиков, робастный перебор подмножеств.
- Оценка TDOA из аудиоокон через GCC-PHAT.
- Локализация в режимах direction и position.
- Сглаживание во времени: EMA азимута/угла места, отбраковка скачков, угловая скорость, агрегированная уверенность.
- JSON-вывод результата и CSV-журнал по окнам.
- Автоматизированный прогон ПМИ с Markdown/JSON/CSV-протоколами.

Важное ограничение

GCC-PHAT уже подключен к файловому конвейеру CSV/WAV -> окна -> предобработка -> TDOA -> локализация. Для реальных записей остаются практические вопросы калибровки: синхронность каналов, известная ориентация направленных микрофонов, отношение сигнал/шум, отражения помещения и подбор частотной полосы.

Подробности:

- docs/PROJECT_OVERVIEW.md - архитектура и конвейер.
- docs/USER_GUIDE.md - команды запуска и форматы файлов.
- docs/ALGORITHM_TDOA_DETAILED.md - математическая логика TDOA, робастность и режимы.
- docs/MODEL_STATUS.md - готовность модулей и ограничения.
- docs/PMI_PROTOCOL_SUMMARY.md - таблица испытаний по ПМИ.

Документация проекта Positioning

Проект positioning - Python-прототип пассивной акустической локализации БПЛА по разности времен прихода сигнала на четыре направленных микрофона. Он нужен для практической части: воспроизводимо сгенерировать или прочитать сигнал, подготовить окна, проверить задержки, получить направление/координаты, сгладить результат во времени и сохранить протоколы.

Назначение

Практическая часть должна показать не только математический решатель, но и рабочий программный конвейер:

```
конфигурация -> сигнал -> окна -> предобработка -> TDOA -> проверка задержек -> локализация  
-> трекинг -> JSON/CSV/ПМИ
```

Текущая реализация закрывает структуру проекта, модели данных, JSON-конфигурацию, синтетический и файловый вход, предобработку, оценку TDOA через GCC-PHAT, робастный TDOA-решатель, локализацию, сопровождение, журналирование и автоматизированный протокол ПМИ.

Структура каталогов

```
positioning/  
  configs/  
    default.json  
  docs/  
    ALGORITHM_TDOA_DETAILED.md  
    MODEL_STATUS.md  
    PMI_PROTOCOL_SUMMARY.md  
    PROJECT_OVERVIEW.md  
    USER_GUIDE.md  
  legacy_cpp/  
    main.cpp  
    validator.cpp  
    validator_using_main.cpp  
  src/  
    positioning/  
      cli.py  
      config.py  
      environment.py  
      geometry.py  
      gcc_phat.py  
      localization.py  
      models.py  
      offline_io.py  
      output.py  
      pmi.py  
      preprocess.py  
      simulation.py  
      tdoa.py  
      tracking.py  
      validation.py  
  tests/  
  tools/  
    demo.py  
    export_docs_to_pdf.py  
    validate.py  
  pyproject.toml  
  README.md
```

src/positioning является единственным активным исходным кодом. legacy_cpp хранит старый C++-вариант только для сравнения и не участвует в запуске.

Основные модули

geometry.py содержит базовые 3D-операции: вектор, скалярное и векторное произведение, норму, расстояние, центрост и решение малой системы 3x3.

`environment.py` рассчитывает скорость звука по температуре и влажности.

`models.py` содержит формальные модели:

- `MicrophoneArray` - четыре микрофона, центр, проверка геометрии, нормаль плоскости;
- `Environment` - параметры среды и скорость звука;
- `AudioFrame` - четырехканальные аудиоданные;
- `PreprocessDiagnostics` - энергия, корреляция и информативность окна;
- `PairDelay` - одна TDOA-пара;
- `TdoaEstimate` - набор TDOA-пар;
- `LocalizationResult` - структурированный результат локализации;
- `AudioConfig`, `LocalizationConfig`, `AppConfig` - конфигурация проекта.

`config.py` загружает и валидирует JSON-конфигурацию.

`simulation.py` генерирует синтетический 4-канальный сигнал по заданной точке источника и сохраняет метаданные сценария. Также модуль умеет генерировать движущийся источник: серия импульсов записывается в один 4-канальный файл, а в `metadata` сохраняется массив `trajectory`.

`offline_io.py` читает и пишет 4-канальные CSV/WAV-файлы и JSON-метаданные.

`preprocess.py` разбивает сигнал на окна, удаляет DC, нормализует каналы, применяет Ханн-окно и частотную маску, рассчитывает диагностику информативности.

`gcc_phat.py` оценивает задержки между парами каналов по алгоритму GCC-PHAT и формирует `TdoaEstimate` для локализатора.

`tdoa.py` проверяет физическую допустимость и транзитивность задержек.

`localization.py` реализует режимы `direction` и `position`, робастную отбраковку ложных пар и расчет доверия.

`tracking.py` сглаживает поток результатов по окнам, отбраковывает резкие низкодостоверные скачки и оценивает скорость изменения направления.

`output.py` сохраняет JSON-результат и CSV-журнал.

`pmi.py` выполняет набор проверок по ПМИ и формирует Markdown/JSON/CSV-протоколы.

`cli.py` объединяет все в команды командной строки.

`tools/export_docs_to_pdf.py` экспортирует README и основные Markdown-документы из `docs` в PDF. PDF складываются в `docs/pdf`, включая общий файл `positioning_documentation.pdf`.

`visualization.py` открывает демонстрационное Tkinter-окно для сравнения исходной и распознанной траектории в режиме потоковой обработки.

Конфигурация

Базовый файл `configs/default.json`:

```
{
  "array": {
    "microphones": [
      {"x": 0.0, "y": 0.0, "z": 0.0},
      {"x": 0.2, "y": 0.0, "z": 0.0},
      {"x": 0.2, "y": 0.2, "z": 0.0},
      {"x": 0.0, "y": 0.2, "z": 0.0}
    ]
  },
  "audio": {
    "sample_rate": 48000,
    "window_size": 2048,
```

```

    "overlap": 0.5,
    "frequency_band": [100.0, 5000.0]
  },
  "environment": {
    "temperature_c": 20.0,
    "humidity_percent": 0.0
  },
  "localization": {
    "mode": "position"
  }
}

```

Проверяются следующие ошибки:

- файл не найден или JSON синтаксически неверен;
- не хватает обязательных секций;
- задано не четыре микрофона;
- микрофоны вырождены или не дают рабочую плоскость;
- частота дискретизации, размер окна или частотная полоса некорректны;
- overlap не лежит в допустимом диапазоне;
- температура/влажность дают некорректную среду;
- режим локализации не равен direction или position.

Конвейер работы

1. Демонстрация

Команда `positioning demo` строит синтетические времена прихода от фиксированной цели и передает их в решатель. Это быстрый способ проверить, что конфигурация, скорость звука и локализация работают.

2. Синтетический сигнал

Команда `positioning generate-synthetic` создает 4-канальный CSV или WAV. Источник задается координатами, частотой, амплитудой и временем излучения. Дополнительно включаются шум, выброс и отраженный путь.

Рядом сохраняется файл метаданных. Он нужен для воспроизводимости: в нем указаны параметры сценария, координаты цели и расчетные времена прихода.

3. Предобработка

Команда `positioning preprocess` читает CSV/WAV, разбивает сигнал на окна, обрабатывает каждое окно и сохраняет обработанные окна в отдельные CSV-файлы. В `preprocess.meta.json` попадает список окон и диагностика качества.

Визуализация привязана к этапу локализации: команда `positioning localization`

`--show-visualization` открывает окно с микрофонной решеткой, исходной точкой или траекторией из `metadata` и распознанными оценками по окнам. `preprocess` не считает координаты и не открывает графические окна. Если `metadata` содержит только одну точку `target`, визуализация ограничивается первым окном и выводит предупреждение. Это защищает демонстрацию от ситуации, когда шумовые оценки по пустым окнам выглядят как ложная траектория.

4. Локализация

Локализация работает с `TdoaEstimate`. В файловом режиме этот объект формируется из предобработанных аудиоокон через GCC-PHAT. В синтетических тестах он также может формироваться напрямую из известных времен прихода.

5. Сопровождение

Команда `positioning track-demo` имитирует поток окон, применяет сглаживание и пишет CSV-журнал. В журнале видны статусы, углы, доверие, скачки, невязки и TDOA-пары.

6. ПМИ

Команда `positioning pmi` запускает набор проверок и формирует:

- ПРОТОКОЛ_ПМИ.md;
- `pmi_results.json`;
- `pmi_results.csv`.

Последний прогон закрыл 15 из 15 проверок.

Статусы результата

`LocalizationResult.status` принимает диагностические значения:

- `ok` - результат рассчитан и проходит проверки;
- `low_confidence` - решение найдено, но качество недостаточно высокое;
- `invalid_input` - входные данные некорректны;
- `insufficient_signal` - сигнал или окно недостаточно информативны.

Статус не заменяет численные метрики. Для анализа нужно смотреть также `confidence`, `residual_error2`, `used_pairs`, `rejected_pairs` и `reliable`.

Что считается готовым результатом практической части

Готовым результатом текущей версии является воспроизводимый стенд, который:

- запускается одной командой;
- использует JSON-конфигурацию;
- принимает или генерирует четыре канала;
- сохраняет метаданные;
- выполняет предобработку;
- проверяет TDOA на физическую согласованность;
- устойчив к единичному ложному пику;
- выдает направление/координаты в допустимых режимах;
- сглаживает последовательность оценок;
- формирует JSON/CSV-логи;
- автоматически закрывает таблицу ПМИ.

4.1. GCC-PHAT из аудио

Команды `positioning localization` и `positioning localize-audio` читают CSV/WAV, делят сигнал на окна, выполняют предобработку для диагностики информативности, оценивают TDOA через GCC-PHAT, передают задержки в локализатор и пишут CSV/JSON. `localization` дополнительно удобно использовать для демонстрации визуализации:

```
positioning localization --input .\data\synthetic\signal.csv --mode direction --show-visualization
```

Для каждой пары микрофонов сохраняются задержка, качество пика и вес. Дальше эти пары проходят существующие проверки физической допустимости, транзитивности и робастного согласования.

Что еще требует проверки на реальном макете

Алгоритмический блок GCC-PHAT реализован, но для реальной аппаратной демонстрации нужны практические проверки:

- синхронность четырех каналов записи;
- отсутствие неизвестных аппаратных задержек между каналами;
- ориентация направленных микрофонов относительно рабочей полусферы;
- подбор частотной полосы под звук БПЛА;
- устойчивость в помещении с отражениями;
- достаточное отношение сигнал/шум.

Руководство пользователя

Документ описывает, как установить проект, запустить основные сценарии, какие файлы получаются на выходе и как интерпретировать результат.

Установка

Перейти в папку проекта:

```
cd "G:\BIG DZ ARCHIEVE\archive dz 8Sem\Диплом\positioning"
```

Установить пакет в editable-режиме с тестовыми зависимостями и экспортом документации:

```
python -m pip install -e ".[dev,docs]"
```

Проверить установку:

```
positioning demo --config .\configs\default.json  
python -m pytest
```

Конфигурация

По умолчанию используется configs/default.json. В нем задаются:

- координаты четырех микрофонов;
- частота дискретизации;
- размер окна;
- перекрытие окон;
- частотная полоса;
- температура;
- влажность;
- режим локализации.

Чтобы проверить влияние температуры, достаточно изменить environment.temperature_c и снова запустить positioning demo. Скорость звука будет пересчитана автоматически.

Чтобы переключить дальнепольный режим, изменить:

```
"localization": {  
  "mode": "direction"  
}
```

Демонстрационный запуск

```
positioning demo --config .\configs\default.json
```

Команда выводит:

- статус;
- режим;
- надежность;
- доверие;
- координаты или направление;
- азимут и угол места;
- невязку TDOA;
- число использованных и отброшенных пар;

- скорость звука.

Сохранить результат в JSON:

```
positioning demo --json-output .\data\logs\demo_result.json
```

Синтетический 4-канальный сигнал

Создать CSV-сигнал:

```
positioning generate-synthetic --output .\data\synthetic\signal.csv
```

Для демонстрации GCC-PHAT лучше использовать широкополосный импульс:

```
positioning generate-synthetic --output .\data\synthetic\signal.csv --source-kind pulse
```

Создать WAV-сигнал:

```
positioning generate-synthetic --format wav --output .\data\synthetic\signal.wav --source-kind pulse
```

Проверить прием WAV и метаданных для пункта 2 ПМИ:

```
positioning inspect-audio --input .\data\synthetic\signal.wav --require-metadata
```

Ожидаемый вывод содержит:

- Статус: ok;
- Формат: WAV;
- Каналов: 4;
- Частота дискретизации: 48000 Гц;
- Метаданные: найдены;
- координаты цели и скорость звука из sidecar-файла.

Пример с шумом, выбросом и отражением:

```
positioning generate-synthetic `
  --output .\data\synthetic\hard_case.csv `
  --target-x 0.6 `
  --target-y 0.4 `
  --target-z 0.3 `
  --source-kind pulse `
  --noise-std 0.02 `
  --outlier-channel 2 `
  --outlier-time 0.15 `
  --outlier-amplitude 0.8 `
  --reflection-delay 0.002 `
  --reflection-gain 0.25
```

Рядом с сигналом создается JSON-файл метаданных. Например для signal.csv будет создан signal.meta.json.

Синтетическая траектория

Для визуальной демонстрации потоковой обработки нужен не один неподвижный источник, а движущаяся цель. Такой файл создается командой:

```
positioning generate-trajectory `
  --output .\data\synthetic\trajectory_demo.csv `
  --duration 1.2 `
  --point-count 12 `
  --noise-std 0.001
```

В результате создаются:

- trajectory_demo.csv - четырехканальный сигнал с серией импульсов;
- trajectory_demo.csv.meta.json - metadata с массивом trajectory.

Каждый элемент trajectory содержит:

- timestamp - время излучения импульса;
- x, y, z - координаты источника в этот момент.

По умолчанию траектория идет от точки (0.35, 0.25, 0.3) м к точке (0.9, 0.55, 0.3) м.

Координаты можно изменить параметрами --start-x, --start-y, --start-z, --end-x, --end-y, --end-z.

Формат CSV-сигнала

CSV содержит четыре числовых столбца: по одному каналу на микрофон. Частота дискретизации берется из конфигурации. Если каналов меньше или больше четырех, чтение завершается понятной ошибкой.

Формат WAV-сигнала

WAV должен быть четырехканальным. Данные приводятся к внутреннему представлению AudioFrame. Этот режим нужен для офлайн-проверок и будущей работы с реальными записями.

Предобработка

Запуск:

```
positioning preprocess --input .\data\synthetic\signal.csv --output-dir .\data\processed
```

Для WAV:

```
positioning preprocess --input .\data\synthetic\signal.wav --output-dir .\data\processed_wav
```

На выходе создаются:

- window_0000.csv, window_0001.csv и так далее;
- preprocess.meta.json с параметрами обработки и диагностикой каждого окна.

Окно считается информативным, если оно имеет достаточную энергию и приемлемую межканальную согласованность.

Визуализация потоковой обработки

Для демонстрации пункта 3 ПМИ сначала выполняется чистая предобработка:

```
positioning preprocess `
  --input .\data\synthetic\trajectory_demo.csv `
  --output-dir .\data\processed_demo
```

Затем команда локализации может сразу открыть всплывающее окно с видом сверху:

```
positioning localization `
  --input .\data\synthetic\trajectory_demo.csv `
  --mode position `
  --show-visualization `
  --trajectory-max-windows 45
```

В окне отображаются:

- синие точки - микрофоны решетки;
- зеленая точка или линия - исходная траектория из .meta.json;
- красные точки или линия - распознанные оценки по окнам;
- пунктирные линии - расхождение исходной и распознанной точек.

Если в metadata есть массив trajectory, окно отрисует всю исходную траекторию зеленой линией. Если metadata содержит только одну цель target, программа выводит предупреждение и ограничивает визуализацию первым окном, чтобы не показывать шумовые выбросы как траекторию.

Для одиночной точки используется тот же механизм:

```
positioning localization `
--input .\data\synthetic\signal.csv `
--mode direction `
--max-windows 1 `
--show-visualization
```

Полезные параметры:

- --mode position - считать и рисовать распознанные координаты;
- --mode direction - считать направление и рисовать его как точку на заданной дальности;
- --trajectory-max-windows - ограничить число окон для быстрой демонстрации;
- --trajectory-min-confidence - не показывать низкодостоверные оценки;
- --trajectory-max-error - отбрасывать точки, которые слишком далеко ушли от исходной траектории;
- --trajectory-direction-range - дальность отрисовки для режима direction.

Локализация из аудиофайла через GCC-PHAT

Запуск полного файлового конвейера:

```
positioning localize-audio `
--input .\data\synthetic\signal.csv `
--mode direction `
--max-windows 1 `
--csv-log .\data\logs\audio_localization.csv `
--json-output .\data\logs\audio_final_result.json
```

Для WAV используется та же команда:

```
positioning localize-audio --input .\data\synthetic\signal.wav --mode direction
```

Что делает команда:

1. читает 4-канальный CSV или WAV;
2. разбивает сигнал на окна из JSON-конфигурации;
3. выполняет предобработку каждого окна для диагностики информативности;
4. оценивает задержки для всех шести пар микрофонов через GCC-PHAT с внутренним удалением DC и RMS-нормализацией каналов;
5. отбраковывает физически невозможные и низкокачественные пары;
6. считает направление или координаты;
7. сглаживает поток результатов;
8. сохраняет CSV-журнал и итоговый JSON.

Полезные параметры:

- --mode direction - дальнепольный режим, основной для демонстрации БПЛА;
- --mode position - ближнепольный/расширенный 3D-режим;
- --max-windows 5 - обработать только первые пять окон;
- --interpolation 8 - интерполяция GCC-PHAT для дробной оценки задержки;
- --min-quality 0.2 - минимальное качество TDOA-пары.

CSV-журнал содержит TDOA-пары в поле `tdoa_pairs_json`. Для каждой пары сохраняются задержка, качество и вес.

Валидация решателя

```
positioning validate --config .\configs\default.json
```

Эта команда запускает синтетические сценарии для проверки TDOA-локализации. Она полезна после изменения геометрии решетки, температуры, режима локализации или кода решателя.

Сопровождение цели

```
positioning track-demo `
--csv-log .\data\logs\tracking.csv `
--json-output .\data\logs\final_result.json `
--inject-low-confidence-jump
```

Команда имитирует поток окон. При включенном `--inject-low-confidence-jump` в середину последовательности добавляется резкий низкодостоверный скачок. Трекер должен отбраковать его или снизить влияние на сглаженную оценку.

CSV-журнал содержит:

- индекс окна;
- timestamp;
- статус;
- режим;
- азимут;
- угол места;
- доверие;
- надежность;
- невязку;
- число использованных/отброшенных пар;
- TDOA-пары;
- параметры среды и окна.

ПМИ

Запуск полного набора проверок:

```
positioning pmi --output-dir .\data\pmi --repeat-count 100
```

Результаты:

- data/pmi/ПРОТОКОЛ_ПМИ.md - человекочитаемый протокол;
- data/pmi/pmi_results.json - машинный JSON;
- data/pmi/pmi_results.csv - таблица для отчета.

Проверки включают запуск программы, прием четырех каналов, файловый режим, учет температуры и влажности, фильтрацию шумного сигнала, корректность TDOA, подавление отраженного пути, расчет направления/координат, JSON/CSV-выход, время обработки окна и нештатные ситуации.

Тесты

```
python -m pytest
```

Тесты покрывают:

- геометрию и модели данных;

- расчет скорости звука;
- загрузку корректного и некорректного JSON;
- синтетическую генерацию;
- CSV/WAV вход-выход;
- предобработку;
- локализацию и робастность;
- сопровождение и вывод;
- CLI smoke-тесты;
- ПМИ.

Экспорт документации в PDF

Сгенерировать PDF-файлы:

```
python .\tools\export_docs_to_pdf.py
```

Результаты сохраняются в docs/pdf:

- отдельный PDF для каждого Markdown-документа;
- общий файл `positioning_documentation.pdf`, объединяющий README и основные документы из docs.

Экспорт использует reportlab и системные шрифты Windows с поддержкой кириллицы.

Частые ошибки

Ошибка конфигурации означает, что JSON не прошел валидацию. Проверить нужно число микрофонов, координаты, `sample_rate`, `window_size`, `overlap`, частотную полосу, температуру, влажность и режим.

Ошибка чтения сигнала обычно означает, что файл не найден, CSV/WAV имеет не четыре канала или данные не являются числовыми.

Статус `insufficient_signal` означает, что окно слишком тихое или каналы плохо согласованы. Для синтетики можно увеличить амплитуду или снизить шум, для реального сигнала - проверить запись и настройки полосы.

Статус `low_confidence` означает, что решение найдено, но невязка, качество пар или согласованность недостаточны для уверенного результата.

Алгоритм TDOA-локализации

Документ описывает текущую реализацию практической части: от модели микрофонной решетки и задержек до локализации, робастной фильтрации ложных пар и сглаживания результата во времени.

Исходные допущения

Система использует четыре направленных микрофона, расположенных компланарно. Конфигурация по умолчанию задает квадратную решетку в плоскости $z=0$:

```
m0 = (0.0, 0.0, 0.0)
m1 = (0.2, 0.0, 0.0)
m2 = (0.2, 0.2, 0.0)
m3 = (0.0, 0.2, 0.0)
```

Для обычной ненаправленной компланарной решетки существует неоднозначность относительно стороны плоскости. В этом проекте принято инженерное допущение: микрофоны направлены, поэтому рабочая полусфера известна. В дальнепольном режиме недостающая нормальная компонента направления выбирается в сторону положительной нормали плоскости решетки.

Скорость звука

Скорость звука рассчитывается по температуре и влажности из конфигурации. Это нужно, потому что задержка TDOA переводится в разность расстояний:

```
delta_r_ij = delta_t_ij * c
```

где delta_t_ij - задержка между микрофонами i и j , c - скорость звука. Изменение температуры в JSON-конфиге меняет расчетную скорость и влияет на итоговые координаты/углы.

Представление TDOA

Для четырех микрофонов доступно шесть пар:

```
(0,1), (0,2), (0,3), (1,2), (1,3), (2,3)
```

Каждая пара хранится как PairDelay:

- pair - индексы микрофонов;
- delay - задержка в секундах;
- quality - качество пика или доверие к паре;
- weight - вес пары в оптимизации.

Набор пар объединяется в TdoaEstimate. В файловом режиме этот объект формируется модулем GCC-PHAT из предобработанных аудиоокон. В синтетических тестах он также может создаваться напрямую из известных времен прихода.

GCC-PHAT

GCC-PHAT оценивает задержку между двумя каналами без явного поиска момента прихода. Для пары сигналов x_i и x_j вычисляется взаимный спектр:

```
R_ij(f) = X_i(f) * conj(X_j(f))
```

PHAT-нормировка оставляет фазовую информацию и подавляет влияние амплитуды:

```
R_phat(f) = R_ij(f) / |R_ij(f)|
```

После обратного БПФ максимум корреляционной функции дает задержку:

```
delay_ij = argmax GCC_PHAT_ij / sample_rate
```

В проекте положительная задержка означает, что канал i пришел позже канала j , то есть знак совпадает с моделью:

```
delay_ij = arrival_i - arrival_j
```

Для каждой пары ограничивается физически допустимый диапазон поиска:

```
max_tau_ij = distance(m_i, m_j) / c
```

Качество пары рассчитывается по выраженности основного пика относительно боковых пиков. Чем выше отношение пик/боковой фон, тем выше `quality` и вес пары в локализаторе.

Проверка допустимости задержек

Перед локализацией задержки проходят несколько проверок.

Физическая допустимость:

```
abs(delay_ij) <= distance(m_i, m_j) / c
```

Если задержка больше максимально возможной для расстояния между микрофонами, пара считается ложной.

Порог качества:

```
quality >= min_quality
```

Пары с низким качеством пика не должны разрушать итоговую оценку.

Транзитивность для троек:

```
delay_ij + delay_jk ~= delay_ik
```

Если задержки внутри тройки микрофонов противоречат друг другу, это признак отражения, выброса или неверного пика корреляции.

Робастный режим

Для защиты от одного ложного канала или отраженного пика используется перебор подмножеств пар. Алгоритм выбирает согласованное подмножество, на котором невязка минимальна, а число отброшенных пар явно попадает в результат.

Оценка невязки использует Huber-подобное ограничение влияния больших ошибок: малые ошибки учитываются квадратично, большие не получают неограниченного веса. Это делает решение устойчивее при единичных выбросах.

В результате локализации сохраняются:

- `used_pairs` - сколько пар вошло в решение;
- `rejected_pairs` - сколько пар отброшено;
- `residual_error2` - суммарная остаточная ошибка;
- `confidence` - расчетная уверенность;
- `reliable` - итоговый признак надежности.

Дальнепольный режим `direction`

В дальней зоне источник описывается единичным вектором направления `u`, а не расстоянием. Для пары микрофонов:

```
delay_ij = dot(m_j - m_i, u) / c
```

Задача сводится к оценке вектора `u` по линейной системе. Для компланарной решетки компоненты в плоскости оцениваются из TDOA напрямую. Нормальная компонента восстанавливается из условия единичной длины:

```
u_z = sqrt(1 - u_x^2 - u_y^2)
```

Знак выбирается в сторону рабочей нормали плоскости, что соответствует направленным микрофонам.

После оценки вектора рассчитываются углы:

```
azimuth = atan2(u_y, u_x)
elevation = atan2(u_z, sqrt(u_x^2 + u_y^2))
```

Этот режим корректен для БПЛА на расстоянии, когда одиночная компактная решетка уверенно определяет направление, но не должна обещать точную дальность.

Ближнепольный режим position

В режиме position решатель оценивает 3D-точку источника. Для пары микрофонов:

```
residual_ij = (|x - m_i| - |x - m_j|) - delay_ij * c
```

Целевая функция:

```
E(x) = sum(weight_ij * residual_ij^2)
```

Текущий решатель использует мультистартовую инициализацию и итерационное уточнение. Этот режим полезен для допустимых ближнепольных сценариев и синтетической проверки, но для одиночной плоской решетки дальность физически менее устойчива, чем направление.

Синтетические сигналы

Генератор формирует четырехканальный сигнал по заданной точке источника. Для каждого микрофона рассчитывается время прихода:

```
t_i = emission_time + distance(source, m_i) / c
```

В сигнал могут добавляться:

- белый шум;
- амплитудное ослабление по расстоянию;
- одиночный выброс в выбранном канале;
- отраженный путь с задержкой и коэффициентом усиления.

Рядом с сигналом сохраняются метаданные: координаты источника, времена прихода, параметры шума, отражения и конфигурация.

Предобработка

Предобработка работает с окнами фиксированного размера:

1. Проверяется наличие четырех каналов.
2. Сигнал разбивается на окна с заданным перекрытием.
3. Из каждого канала удаляется постоянная составляющая.
4. Каналы нормализуются по амплитуде.
5. Применяется Ханн-окно.
6. В частотной области применяется полосовая маска.
7. Рассчитываются энергия и межканальная согласованность.

Если энергия слишком мала или каналы плохо согласованы, окно получает диагностический статус недостаточной информативности.

Сглаживание во времени

Модуль сопровождения хранит историю результатов по окнам и применяет экспоненциальное сглаживание азимута и угла места. Для каждого нового окна:

- проверяется статус локализации;

- резкий скачок при низкой уверенности отбраковывается;
- обновляется сглаженный азимут и угол места;
- оценивается скорость изменения направления;
- пересчитывается агрегированная уверенность по истории.

Так система выдает последовательность устойчивых оценок, а не независимые скачущие измерения.

Выходные данные

Один результат может быть сохранен в JSON. Пооконный журнал сохраняется в CSV и содержит:

- номер окна и время;
- режим локализации;
- статус и сообщение;
- координаты или направление;
- азимут и угол места;
- доверие и надежность;
- невязку;
- число использованных и отброшенных пар;
- задержки по парам;
- параметры среды и окна.

Эти файлы используются для отчета, визуализации, отладки и закрытия проверок ПМИ.

Текущее ограничение

Проект уже имеет офлайн-вход, синтетику, предобработку и GCC-PHAT. Для реального аппаратного макета требуется дополнительная калибровка каналов и проверка на реальных WAV/PCM-записях: синхронность каналов, аппаратные задержки, отражения помещения и подбор частотной полосы.

Состояние модели и готовность проекта

Документ фиксирует фактическое состояние Python-проекта после реализации этапов 1-4 и 6-10.

Общая готовность

Активная версия проекта находится в `src/positioning`. Старый C++-код перенесен в `legacy_cpp` и больше не является основной веткой разработки.

Проект собирается и запускается как Python-пакет. Основные команды:

```
python -m pip install -e ".[dev]"
positioning demo --config .\configs\default.json
positioning validate --config .\configs\default.json
positioning pmi --output-dir .\data\pmi --repeat-count 100
python -m pytest
```

Реализованные этапы

Этап 1. Рабочая Python-структура

Готово:

- активный код вынесен в пакет `src/positioning`;
- CLI-точка входа находится в `src/positioning/cli.py`;
- вспомогательные обертки оставлены в `tools/demo.py` и `tools/validate.py`;
- C++-код сохранен только как архивный референс в `legacy_cpp`;
- `__rusache__` не должен отслеживаться;
- добавлен `pyproject.toml` для `editable`-установки;
- `pytest` используется как основной способ проверки.

Этап 2. Модель данных и конфигурация

Готово:

- `MicrophoneArray` проверяет число микрофонов, размерность, невырожденность и компланарность;
- `Environment` хранит температуру, влажность и рассчитанную скорость звука;
- `AudioFrame` описывает четырехканальные аудиоданные;
- `TdoaEstimate` хранит задержки, качества, веса и пары;
- `LocalizationResult` хранит режим, координаты, направление, углы, статус, невязку и диагностику;
- `configs/default.json` задает решетку, аудио, среду и режим локализации;
- загрузчик конфигурации проверяет отсутствующие поля, число микрофонов, частоты, `overlap` и параметры среды.

Этап 3. Офлайн-вход и синтетические сигналы

Готово:

- генератор синтетического 4-канального сигнала по точке источника;
- поддержаны шум, задержки, ослабление, одиночный выброс и отраженный путь;
- реализованы CSV и WAV вход/выход;

- рядом с сигналом сохраняются JSON-метаданные сценария.

Этап 4. Предварительная обработка

Готово:

- разбиение потока на окна фиксированного размера;
- удаление постоянной составляющей;
- нормализация каналов;
- Hann-окно;
- частотная маска по заданной полосе;
- диагностика информативности окна по энергии и межканальной согласованности.

Этап 5. Оценка TDOA из аудио через GCC-PHAT

Готово:

- реализован GCC-PHAT для пары каналов;
- задержка имеет тот же знак, что и модель `arrival_i - arrival_j`;
- поиск пика ограничивается физически допустимым временем по расстоянию между микрофонами;
- рассчитывается качество пары по отношению основного пика к боковому фону;
- для всех шести пар микрофонов формируется `TdoaEstimate`;
- добавлена CLI-команда `positioning localize-audio`;
- файловый конвейер работает как `CSV/WAV -> окна -> предобработка -> GCC-PHAT -> локализация -> трекинг -> CSV/JSON`;
- добавлены unit-тесты GCC-PHAT и smoke-тест CLI.

Этап 6. Подавление ложных задержек

Готово:

- проверка физической допустимости задержек;
- проверка транзитивности задержек для троек микрофонов;
- отбраковка пар с низким качеством;
- робастный перебор подмножеств пар;
- Huber-подобная оценка остаточной ошибки;
- явная диагностика использованных и отброшенных пар.

Этап 7. Локализация направления и координат

Готово:

- режим `direction` для дальней зоны;
- перевод направления в азимут и угол места;
- режим `position` для ближнепольной/расширенной 3D-оценки;
- в результате явно указан режим;
- для 3D-режима выдаются невязка, число использованных пар и признак надежности.

Этап 8. Сглаживание во времени

Готово:

- хранение истории результатов по окнам;
- экспоненциальное сглаживание азимута и угла места;
- отбраковка резких скачков при низкой уверенности;
- оценка скорости изменения направления;
- агрегированная уверенность по нескольким окнам.

Этап 9. Выходные данные и журналирование

Готово:

- JSON-вывод одного результата;
- CSV-журнал по окнам;
- сохранение параметров среды, окна, задержек, невязки, доверия и статуса;
- разделение пользовательского вывода CLI и подробного машинного журнала;
- понятные сообщения для ошибок конфигурации, чтения файлов и неинформативных сигналов.

Этап 10. Испытания по ПМИ

Готово:

- автоматизированный запуск ПМИ через `positioning pmi`;
- Markdown-протокол;
- JSON-результаты;
- CSV-таблица;
- проверка запуска, файлового режима, среды, фильтрации, TDOA, отражений, направления/координат, логов, времени обработки и нештатных ситуаций.

Последний прогон: 15 из 15 проверок пройдено.

Ограничения

Главное оставшееся ограничение - не алгоритмическое, а макетно-измерительное: GCC-PHAT реализован, но для реальных аудиозаписей надо проверить синхронность каналов, аппаратные задержки, ориентацию направленных микрофонов, отражения и отношение сигнал/шум.

Также надо учитывать физику одиночной компактной решетки:

- режим `direction` является основным для дальней зоны;
- режим `position` применим для допустимых ближнепольных сценариев и синтетической проверки;
- точная дальность в дальней зоне по одной плоской решетке не должна заявляться как гарантированный результат.

Следующий приоритет

Следующий инженерный этап - сделать демонстрационный визуальный стенд:

4-канальный файл -> графики сигналов -> пики GCC-PHAT -> TDOA-пары -> направление -> трекинг
-> отчет

Параллельно стоит расширить ПМИ испытаниями на реальных WAV/PCM-файлах и сценариями с несколькими уровнями отношения сигнал/шум.

Протокол ПМИ

Документ фиксирует состав автоматизированных испытаний по ПМИ и результат последнего прогона.

Команда запуска

```
positioning pmi --output-dir .\data\pmi --repeat-count 100
```

Выходные файлы:

- data/pmi/ПРОТОКОЛ_ПМИ.md;
- data/pmi/pmi_results.json;
- data/pmi/pmi_results.csv.

Итог последнего прогона

Пройдено 15 из 15 проверок.

№	Проверка	Статус	Что подтверждает
1	Запуск программы	пройдено	CLI и базовый сценарий локализации запускаются без ручной подготовки
2	Прием четырех каналов и метаданных	пройдено	CSV-сигнал из четырех каналов и sidecar-метаданные корректно записываются и читаются
3	Обработка файлового режима	пройдено	Поддерживаются CSV и WAV входы
4	Учет температуры и влажности	пройдено	Изменение среды меняет скорость звука
5	Фильтрация сигнала с шумом	пройдено	Шумное окно проходит предобработку и получает информативную диагностику
6	Оценка TDOA из аудио через GCC-PHAT	пройдено	GCC-PHAT восстанавливает задержки из синтетического четырехканального импульса
7	Корректность TDOA на синтетике	пройдено	Оценка направления по синтетическим задержкам совпадает с эталоном
8	Подавление отраженного пути	пройдено	Одиночная ложная TDOA-пара отбраковывается
9	Вычисление направления и координат	пройдено	Работают режимы `direction` и допустимый `position`
10	JSON/CSV-выход и журнал	пройдено	Результат и пооконный журнал сохраняются в структурированном виде
11	Время обработки окна	пройдено	Предобработка одного окна быстрее длительности окна
12	Отсутствие одного канала	пройдено	Некорректное число каналов отклоняется понятной ошибкой
13	Некорректные параметры среды	пройдено	Неверные температура/влажность отклоняются
14	Недостаточная информативность сигнала	пройдено	Тихое окно получает статус недостаточной информативности
15	Длительный повторный прогон	пройдено	Повторная последовательная обработка завершается стабильно

Назначение проверок

ПМИ проверяет не отдельную формулу, а готовность практического программного стенда:

- программа запускается из командной строки;
- конфигурация читается из JSON;
- четырехканальный файловый вход работает;
- параметры среды участвуют в расчете;

- сигнал проходит предобработку;
- TDOA извлекается из аудиоокон через GCC-PHAT;
- TDOA-решатель корректен на синтетике;
- ложные задержки не разрушают оценку;
- режимы направления и координат явно различаются;
- выходные данные пригодны для отчета;
- нештатные ситуации не приводят к непонятному traceback;
- длительный повторный прогон не ломает состояние трекера.

Как использовать результаты в отчете

Для отчета удобно включить:

- таблицу из этого документа;
- файл `data/pmi/ПРОТОКОЛ_ПМИ.md` как полный машинно сформированный протокол;
- `pmi_results.csv` как приложение или исходник таблицы;
- `pmi_results.json` как подтверждение структурированного формата результата.

Ограничение протокола

Проверка GCC-PHAT выполняется на синтетическом широкополосном импульсе с известными сдвигами каналов. Для аппаратного макета протокол надо расширить реальными WAV/PCM-записями и калибровкой межканальных задержек.